

Mouhamed Sy
Fatou Boye Ba
Licence 2 cybersecurite
P11 SIMAC

INFRASTRUCTURES CLOUD
Examen

1. Introduction

Dans un contexte où les cyberattaques sont de plus en plus fréquentes et sophistiquées, les entreprises doivent mettre en place des solutions efficaces pour surveiller, détecter et répondre aux incidents de sécurité. Les Security Operations Centers (SOC) jouent un rôle central dans cette stratégie en assurant une supervision continue des systèmes d'information.

Ce projet s'inscrit dans le cadre du module *Cloud et Infrastructures Virtuelles* et a pour objectif de concevoir et déployer un SOC conteneurisé à l'aide de Docker Compose. Il vise à mettre en pratique les concepts de virtualisation, de segmentation réseau, de gestion des logs et de réponse aux incidents à travers l'intégration d'outils tels que Wazuh, Suricata, Grafana et TheHive.

L'environnement simulé repose sur une entreprise fictive nommée **TechCorp SA**, spécialisée dans les services numériques. Dans ce contexte, le SOC déployé permet de surveiller l'infrastructure, détecter des activités malveillantes et gérer efficacement les incidents de sécurité afin de renforcer la posture globale de cybersécurité de l'organisation.

2. Architecture

L'architecture du SOC Lab repose sur une approche conteneurisée basée sur Docker Compose, permettant de déployer et d'orchestrer plusieurs services interconnectés au sein d'un environnement isolé et modulaire.

La stack est composée de cinq principaux services : Wazuh Manager (collecte et analyse des logs), Wazuh Dashboard (visualisation des événements de sécurité), Suricata (système de détection d'intrusion réseau), Grafana (visualisation des données) et TheHive (gestion des incidents de sécurité). Chaque service joue un rôle spécifique dans le processus global de détection et de réponse aux menaces.

Afin de garantir la sécurité et une bonne organisation des communications, l'architecture est segmentée en deux réseaux Docker distincts :

- **soc-frontend** : réseau exposé vers l'extérieur, permettant l'accès aux interfaces utilisateur (Wazuh Dashboard, Grafana, TheHive).
- **soc-backend** : réseau interne, réservé aux communications entre les services (Wazuh Manager, Suricata, etc.).

Cette segmentation permet de limiter l'exposition des services critiques et d'appliquer le principe du moindre privilège.

En ce qui concerne la persistance des données, des volumes Docker nommés ont été configurés pour les services nécessitant la conservation des informations (logs, dashboards, cas d'incidents). Cela garantit que les données ne sont pas perdues en cas de redémarrage des conteneurs.

Suricata, en tant qu'outil de détection réseau, est configuré pour avoir accès à l'interface réseau de l'hôte, ce qui lui permet d'analyser le trafic en temps réel. Cette configuration est essentielle pour assurer une détection efficace des attaques.

Les flux de données au sein de l'architecture suivent une logique claire : Suricata analyse le trafic réseau et génère des alertes, Wazuh centralise et corrèle les logs, Grafana permet leur visualisation, et TheHive prend en charge la gestion des incidents détectés.

Ainsi, cette architecture modulaire, segmentée et orientée sécurité constitue une base solide pour un SOC fonctionnel, tout en restant flexible et évolutive.

3. Déploiement

Le déploiement du SOC Lab a été réalisé à l'aide de Docker Compose, permettant d'orchestrer plusieurs services de manière simple et reproductible. L'ensemble de la stack comprend Wazuh, Suricata, Grafana et TheHive, tous connectés via des réseaux Docker dédiés.

Lors de la mise en place, plusieurs difficultés techniques ont été rencontrées. Tout d'abord, les versions des images initialement recommandées dans les consignes se sont révélées incompatibles ou instables dans notre environnement. Certains services ne démarraient pas correctement ou présentaient des erreurs de configuration. Il a donc été nécessaire d'identifier et d'utiliser des versions alternatives plus stables, ce qui a demandé des recherches et des tests supplémentaires.

Par ailleurs, la configuration du paramètre système `vm.max_map_count` a été indispensable pour assurer le bon fonctionnement des composants liés à Wazuh. Sans cet ajustement, certains services restaient en échec au démarrage.

Une autre contrainte importante concerne la lourdeur de la stack déployée. En effet, l'exécution simultanée de plusieurs services de cybersécurité consomme rapidement des ressources (CPU, mémoire). Cela nécessite une machine disposant de capacités suffisantes pour garantir la stabilité et la performance de l'ensemble. Sur des machines peu puissantes, des ralentissements ou des échecs de démarrage peuvent survenir.

Ensuite, la gestion des réseaux Docker a nécessité une attention particulière. Une segmentation a été mise en place entre un réseau frontend (accessible depuis l'extérieur) et un réseau backend (réservé aux communications internes), afin de respecter les bonnes pratiques de sécurité. Des tests de connectivité ont été réalisés pour vérifier l'isolation entre les services.

La configuration des volumes a également été essentielle pour garantir la persistance des données, notamment pour Wazuh, Grafana et TheHive. Cela permet de conserver les logs, dashboards et informations d'incidents même après redémarrage des conteneurs.

Par ailleurs, l'utilisation d'un fichier `.env` a permis d'externaliser les variables sensibles telles que les mots de passe et les ports, évitant ainsi leur exposition dans le fichier `docker-compose.yml`.

Enfin, après résolution des différents problèmes, la stack a été déployée avec succès à l'aide de la commande :

```
docker compose up -d
```

Tous les services ont pu atteindre un état fonctionnel, permettant ainsi de passer aux phases de test et de simulation d'attaques.

4. Résultats des attaques

Cette partie présente les résultats obtenus lors des trois phases de simulation d'attaque, en mettant en évidence la chronologie des événements (timeline), les alertes générées ainsi que leur analyse.

Phase 1 : Scan réseau (Reconnaissance)

La première phase a consisté en un scan réseau à l'aide de Nmap afin d'identifier les ports ouverts sur l'infrastructure.

- **Timeline :**
 - Lancement du scan Nmap (ports 1-10000)
 - Détection quasi immédiate par Suricata
 - Génération d'une alerte dans Wazuh
- **Résultats observés :**
 - Règle déclenchée : détection de scan réseau
 - Niveau de sévérité : moyen à élevé ($\approx$ level 10)
 - Signature Suricata : Nmap SYN Scan
- **Analyse :**

Le SOC a correctement identifié une activité de reconnaissance. La rapidité de détection montre une bonne efficacité du système..

Phase 2 : Attaque Brute Force SSH

Cette phase visait à simuler une attaque par force brute sur un service SSH vulnérable.

- **Timeline :**
 - Lancement de l'attaque avec Hydra
 - Multiples tentatives de connexion échouées
 - Déclenchement d'une alerte Wazuh après plusieurs essais
 - Activation de la réponse automatique (blocage IP)
- **Résultats observés :**
 - Alerte Wazuh niveau 10 (attaque brute force)
 - Détection après plusieurs tentatives (seuil configuré)
 - Blocage automatique de l'adresse IP via iptables
- **Analyse :**

Le mécanisme de détection est efficace et permet une réponse rapide grâce à l'active response. Cela montre la capacité du SOC à non seulement détecter mais aussi réagir automatiquement aux attaques.

Phase 3 : Injection SQL

La dernière phase a consisté en une attaque par injection SQL sur une application web vulnérable (DVWA).

- **Timeline :**
 - Accès à l'application DVWA
 - Injection SQL via le formulaire de connexion
 - Détection par Suricata
 - Création d'un incident dans TheHive
- **Résultats observés :**
 - Alerte Suricata détectant une signature
 - Logs visibles dans Wazuh
 - Incident créé dans TheHive
- **Analyse :**

Cette phase met en évidence la capacité du SOC à détecter des attaques applicatives plus complexes. L'intégration avec TheHive permet une gestion structurée de l'incident selon un cycle clair (identification, confinement, etc.).

Conclusion des résultats

L'ensemble des attaques simulées a été détecté avec succès par les outils du SOC. La corrélation des logs, la visualisation et la gestion des incidents démontrent une architecture fonctionnelle et cohérente.

5. Analyse de sécurité

L'analyse des résultats obtenus lors des différentes simulations d'attaques met en évidence plusieurs points forts ainsi que certaines limites du SOC déployé.

Parmi les points forts, on note une bonne capacité de détection des activités malveillantes. Les scans réseau effectués ont été rapidement identifiés par Wazuh, avec des alertes pertinentes et correctement classifiées. De même, les tentatives de brute force SSH ont généré des alertes de niveau élevé, accompagnées d'une réponse active permettant de bloquer automatiquement l'adresse IP attaquante. L'intégration de Suricata a également permis de détecter efficacement des attaques plus complexes, telles que les injections SQL.

Un autre point positif réside dans la centralisation et la corrélation des logs. Grâce à Wazuh et Grafana, il est possible d'avoir une vue globale des événements de sécurité, facilitant ainsi l'analyse et la prise de décision. Par ailleurs, l'utilisation de TheHive permet une gestion structurée des incidents, avec un suivi clair des différentes étapes de réponse (identification, confinement, éradication, récupération).

Cependant, certaines limites ont été observées. Le SOC peut générer des faux positifs, notamment lors des scans réseau, ce qui peut entraîner une surcharge d'alertes et compliquer l'analyse. De plus, la configuration par défaut de certains outils reste basique et nécessite un ajustement fin des règles pour améliorer la précision de détection.

Enfin, l'absence de mécanismes avancés d'automatisation et de threat intelligence limite la capacité du SOC à anticiper certaines menaces. Une amélioration de ces aspects permettrait de rendre le système plus proactif face aux attaques.

En conclusion, le SOC mis en place constitue une base solide et fonctionnelle, mais nécessite des optimisations pour atteindre un niveau de sécurité plus avancé et adapté à un environnement de production.

6. Recommandations

Afin d'améliorer la sécurité et la robustesse du SOC déployé, plusieurs recommandations peuvent être proposées.

Tout d'abord, la gestion des secrets doit être renforcée. L'utilisation de fichiers `.env` constitue une bonne pratique, mais en environnement de production, il serait préférable d'adopter une solution dédiée comme Docker Secrets ou un gestionnaire de secrets externe (ex : HashiCorp Vault). Cela permettrait de sécuriser davantage les mots de passe, clés API et autres informations sensibles.

Ensuite, le durcissement des conteneurs est essentiel. Il est recommandé d'exécuter les conteneurs avec des utilisateurs non privilégiés (non-root), de limiter les permissions système (capabilities), et d'utiliser des systèmes de fichiers en lecture seule lorsque cela est possible. Ces mesures réduisent considérablement la surface d'attaque en cas de compromission d'un service.

Enfin, la supervision et la gestion des alertes peuvent être améliorées. Il serait pertinent d'intégrer un système de notification automatique (email, Slack, etc.) afin d'alerter rapidement les administrateurs en cas d'incident critique. De plus, la mise en place de mécanismes d'escalade permettrait d'assurer une réponse rapide et adaptée selon la gravité des incidents détectés.

7. Conclusion

Ce projet a permis de mettre en œuvre un SOC conteneurisé complet en utilisant Docker Compose, en intégrant plusieurs outils essentiels à la cybersécurité tels que Wazuh, Suricata, Grafana et TheHive. Il a ainsi offert une expérience concrète de déploiement d'une architecture multi-services sécurisée, avec gestion des réseaux, des volumes et des configurations sensibles.

À travers les différentes simulations d'attaques (scan réseau, brute force et injection SQL), nous avons pu observer l'efficacité des mécanismes de détection et de réponse aux incidents. L'utilisation combinée des outils a permis une meilleure visibilité sur les événements de sécurité et une gestion structurée des incidents.

Ce projet a également renforcé des compétences clés en conteneurisation, en analyse de logs et en sécurité des systèmes. À l'avenir, cette infrastructure pourrait être améliorée par l'intégration de solutions de threat intelligence, l'automatisation des alertes ou encore un déploiement en environnement cloud pour une utilisation en conditions réelles.